

Error 1 & 2: AttributeError: '_GeneratorContextManager' object has no attribute 'query'

The code have been Using @contextmanager

@contextmanager

```
def get_default_db_session():
    db = Session(bind=engine)
    try:
        yield db
    finally:
        db.close()
```

When FastAPI uses Depends(get_default_db_session), it gets a CONTEXT MANAGER NOT the actual Session object!

The Problem:

- @contextmanager creates a context manager object (something you use with with)
- FastAPI's Depends() expects a generator function (with yield)
- Context manager ≠ Generator function
- So db was a context manager, not a Session
- Context managers don't have .query(), .add(), .commit() methods

solution

Use Plain generator function (remove @contextmanager)

```
def get_default_db_session():
    db = Session(bind=engine)
    try:
        yield db # FastAPI gets the Session object
    finally:
        db.close()
```

Now FastAPI's Depends() gets the actual Session

GET

/v1/tenant/ Get Current Tenant

Get the current tenant

Parameters

No parameters

Execute

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/v1/tenant/' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/v1/tenant/
```

Server response

Code	Details
500	Error: Internal Server Error
<i>Undocumented</i>	Response body Internal Server Error

Error 3, 4 & 5: AttributeError: 'TenantInfo' object has no attribute 'current_status'

The Pydantic schema **TenantInfo** didn't match SQLAlchemy model (**Tenant**).

Schema did not have status while model have

```
class TenantInfo(BaseModel):
    tenant_schema_name: str
    sub_domain: str
```

model

```

class Tenant(Base):
    __tablename__ = "tenants"
    __table_args__ = ({ "schema": "shared"},)

    tenant_id = sa.Column(
        "tenant_id",
        sa.Integer,
        primary_key=True,
        nullable=False,
        comment="The unique identifier for the tenant",
    )

    tenant_schema_name = sa.Column(
        "tenant_schema_name",
        sa.String(256),
        nullable=False,
        unique=True,
        comment="The schema for the database of the tenant example:
sitel",
    )

    sub_domain = sa.Column(
        "sub_domain",
        sa.String(256),
        nullable=False,
        index=True,
        unique=True,
        comment="the name of the tenant example: sitel",
    )

    current_status = sa.Column(
        "current_status",
        # inherit_schema=True will place the enum in the correct schema
and not in the public schema
        sa.Enum(TenantStatuses, inherit_schema=True),
        nullable=False,
        comment="The status of the tenant",
    )

```

The Problem:

- Model has 3 fields
- Schema has only 2 fields
- When you tried to access `tenant_data.current_status`, it didn't exist!

Solution

Add status in schema

```
from pydantic import BaseModel

from typing import Literal

status=Literal["active", "inactive"]

class TenantInfo(BaseModel):
    tenant_schema_name: str
    sub_domain: str
    current_status: status
```

POST /v1/tenant/ Create Tenant Endpoint

Create a new tenant

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "tenant_schema_name": "string",
  "sub_domain": "string"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/v1/tenant/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "tenant_schema_name": "string",
    "sub_domain": "string"
  }'
```

Request URL

http://127.0.0.1:8000/v1/tenant/

Server response

Code	Details
------	---------

500	Error: Internal Server Error
-----	------------------------------

Undocumented

Response body

Internal Server Error

Response headers

```
content-length: 21
content-type: text/plain; charset=utf-8
date: Wed, 18 Mar 2026 08:58:42 GMT
server: uvicorn
```

Error 6: InvalidTextRepresentation: invalid input value for enum
shared.status: "good"

In tenant model (tenant.py)

```
class TenantStatuses(Enum):  
    active = "active"  
    inactive = "inactive"
```

In database, PostgreSQL created an ENUM type with ONLY these values:

ENUM('active', 'inactive')

While in schema file

```
class TenantInfo(BaseModel):  
    tenant_schema_name: str  
    sub_domain: str  
    current_status: str
```

I as a User when send other than active and inactive :

```
{  
    "current_status": "good" }
```

It will cause error so do this:

Use literals

```
from pydantic import BaseModel  
  
from typing import Literal  
  
status=Literal["active", "inactive"]  
  
class TenantInfo(BaseModel):  
    tenant_schema_name: str  
    sub_domain: str  
    current_status: status
```